

Skid-Steer Robot

Steering Control Algorithm

Implementation & Verification Specification

Version	1.0
Status	Draft — for developer review
Date	April 2026

1. System Overview

This document specifies the steering control algorithm for a 4-wheel Skid-Steer robot. Its purpose is to translate operator joystick commands into correct per-motor CAN throttle commands, while preventing rollover and providing smooth motion.

- Configuration: 4-wheel Skid-Steer, two motor controllers (Left / Right)
- CAN protocol: DriveCommand at 0x300+NodeID, SpeedTelemetry at 0x310+NodeID
- Operating mode: Speed Mode (MotorMode = 01 in DriveCommand)
- Guiding principle: Operator-requested linear speed is sacred. Turn radius is adapted to prevent rollover. Only in a physical impossibility is speed reduced to honour the requested turn angle.

2. Inputs & Parameters

2.1 Per-Loop Inputs

Signal	Range	Description
DriveCmd	-1.0 ... +1.0	Drive request. +1 = full forward, -1 = full reverse
SteerCmd	-1.0 ... +1.0	Turn request. +1 = right, -1 = left
V_left_telem	m/s	Actual left wheel speed (converted from RPM telemetry)
V_right_telem	m/s	Actual right wheel speed (converted from RPM telemetry)
dt	s	Loop cycle time in seconds

2.2 System Parameters (to fill before deployment)

Parameter	Unit	Default	Notes
W	m	(measure)	Track width — distance between left and right wheel contact centres
r	m	(measure)	Wheel radius
V_max	m/s	(measure)	Maximum robot speed. Throttle 1000 maps to this value
a_lat_max	m/s ²	3.0	Max lateral acceleration before rollover risk
MaxAccel	m/s ²	2.0	Max allowed acceleration / deceleration rate (slew limiter)
V_pivot	m/s	0.2	Speed below which pivot logic applies (avoids divide-by-zero)
V_blend_end	m/s	1.0	Speed above which full dynamic logic applies
deadzone	—	0.05	Joystick dead-band threshold (suppress noise when stick released)
telem_timeout	s	0.2	If no SpeedTelemetry received within this window, emergency stop

3. Unit Conversions

RPM → m/s

$$V \text{ [m/s]} = \text{RPM} \times 2\pi \times r / 60$$

m/s → CAN Throttle

```
Throttle = round( (V / V_max) × 1000 )
Throttle = clamp(Throttle, -1000, +1000)
```

CAN accepts integer throttle in the range [-1000, +1000]. The final clamp is a numerical safety guard only — if the algorithm is correct the value should already be within bounds.

4. Algorithm — Six Stages

Execute every control loop cycle in the order below.

Stage 1 — Reference Speed

```
V_robot_actual = (V_left_telem + V_right_telem) / 2
V_target_drive = DriveCmd × V_max
V_for_omega    = max( |V_robot_actual|, |V_target_drive| ) //
Option Y
```

V_{for_omega} is the reference used for anti-rollover calculations in Stage 2. Taking the maximum of actual and commanded speed ensures the turn limit is computed conservatively when the robot is already moving faster than the current command.

Stage 2 — TurnAmount by Speed Zone

Three zones based on V_{for_omega} :

Zone A — Pivot ($V_{for_omega} \leq V_{pivot}$)

At near-zero speed there is no centrifugal force. TurnAmount equals maximum differential directly:

```
TurnAmount = SteerCmd × V_max
```

Zone B — Dynamic ($V_{for_omega} \geq V_{blend_end}$)

Anti-rollover constraint: lateral acceleration must not exceed a_{lat_max} .

```
ω_max = a_lat_max / V_for_omega
TurnAmount = SteerCmd × (ω_max × W / 2)
```

Derivation: $a_{lat} = V \times \omega \leq a_{lat_max} \Rightarrow \omega \leq a_{lat_max} / V$. The wheel speed difference equals $\omega \times W$, so each side contributes $\omega \times W/2$.

Zone C — Blend ($V_{pivot} < V_{for_omega} < V_{blend_end}$)

Linear interpolation between pivot and dynamic results:

```
ratio = (V_for_omega - V_pivot) / (V_blend_end - V_pivot)
// ∈ (0, 1)
TurnAmount_pivot = SteerCmd × V_max
ω_max_blend = a_lat_max / V_blend_end
TurnAmount_dynamic = SteerCmd × (ω_max_blend × W / 2)
TurnAmount = (1 - ratio) × TurnAmount_pivot + ratio ×
TurnAmount_dynamic
```

Stage 3 — Conflict Resolution

Principle: preserve the turn, sacrifice linear speed.

```
peak_side = |V_target_drive| + |TurnAmount|
if peak_side > V_max:
    V_target_drive = sign(V_target_drive) × (V_max - |TurnAmount|)
```

After this step, both motor targets computed in Stage 4 are guaranteed to lie within $[-V_{\max}, +V_{\max}]$.

Stage 4 — Per-Motor Targets

```
Target_left  = V_target_drive + TurnAmount
Target_right = V_target_drive - TurnAmount
```

Sign convention: $\text{SteerCmd} > 0$ is a right turn. Left wheel accelerates, right wheel decelerates (or reverses in pivot).

Stage 5 — Slew Rate Limiter

Prevents instantaneous torque jumps. Applied independently to each side.

```
max_delta = MaxAccel * dt

delta_left = clamp(Target_left - Cmd_left_prev, -max_delta,
+max_delta)
Cmd_left   = Cmd_left_prev + delta_left

delta_right = clamp(Target_right - Cmd_right_prev, -max_delta,
+max_delta)
Cmd_right   = Cmd_right_prev + delta_right
```

Cmd_left and Cmd_right become the new previous values for the next cycle.

Stage 6 — CAN Throttle Output

```
Throttle_left  = round( (Cmd_left / V_max) * 1000 )
Throttle_right = round( (Cmd_right / V_max) * 1000 )
```

Send via DriveCommand frame (CAN ID $0x300 + \text{NodeID}$) with MotorMode = 01 (Speed Mode).

5. Safeguards

5.1 Telemetry Timeout

If SpeedTelemetry has not been received from either side within telem_timeout seconds:

- Set DriveCmd = 0 and SteerCmd = 0 for this cycle
- Log / raise a fault flag
- Slew Rate Limiter remains active — the robot decelerates smoothly to zero

5.2 Initialisation

Cmd_left_prev and Cmd_right_prev must be initialised to the first telemetry reading received, not to 0. Initialising to 0 when the robot is already moving causes a large initial delta that the slew limiter cannot suppress in one cycle.

5.3 Final Output Clamp

```
Throttle_left = clamp(Throttle_left, -1000, +1000)
Throttle_right = clamp(Throttle_right, -1000, +1000)
```

This is purely a numerical safety guard. Under normal operation the values should be in range before clamping.

5.4 Joystick Dead-Band (optional)

If $|DriveCmd| < \text{deadzone}$, treat DriveCmd as 0. If $|SteerCmd| < \text{deadzone}$, treat SteerCmd as 0. This eliminates stick noise at rest.

6. Out of Scope

The following are not part of this algorithm. Implement them in a separate layer:

- Loop timing and scheduling
- CAN frame reading and parsing
- CAN frame transmission
- FaultLevel and AlarmsFaultsRpt management
- RPM-to-m/s conversion (provide already-converted values as inputs)

7. Developer Verification Checklist

Test Case	Expected Result	Pass ✓
DriveCmd=0, SteerCmd=0 → robot stationary	Throttle_left = Throttle_right = 0	
DriveCmd=1, SteerCmd=0 → full forward straight	Throttle_left = Throttle_right = 1000 (after slew ramp)	
DriveCmd=0, SteerCmd=+1 → right pivot	Throttle_left = +1000, Throttle_right = -1000	

DriveCmd=0, SteerCmd=-1 → left pivot	Throttle_left = -1000, Throttle_right = +1000	
DriveCmd=1, SteerCmd=1 at $V_{\text{for_omega}} < V_{\text{pivot}}$ → conflict	Drive capped; Throttle never exceeds 1000 on either side	
DriveCmd=1 at $V_{\text{for_omega}} = V_{\text{max}}$ → dynamic zone: full steer	TurnAmount limited by $a_{\text{lat_max}} / V_{\text{max}}$; no overflow	
Instant DriveCmd=1 → 0 with $dt=0.01$ s	Cmd drops by $\text{MaxAccel} \times dt$ per step, not instantly	
Telem timestamp > 200 ms old → timeout	Fault flag raised; robot decelerates to stop via slew	
First telem = 1.5 m/s → Cmd_prev initialised correctly	No large first-cycle delta; smooth startup	
DriveCmd = 0.04 (below deadzone) → treated as 0	$V_{\text{target_drive}} = 0$	

End of Document